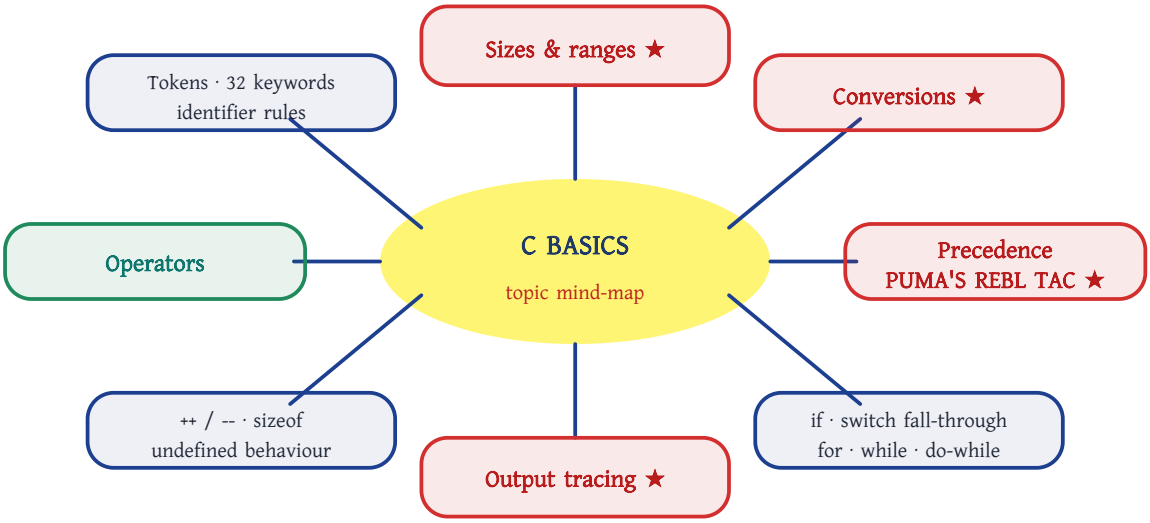
C Data Types & Operators

Sizes · conversions · precedence · if, switch, loops · exam-ready in 11 sheets



↑ the whole lesson on one page: red = high-yield, green = the family every question draws on

Syllabus coverage

The syllabus line is "**Structured programming language: data types, operators, expressions, control structures**". Functions, scope, storage classes and recursion; the preprocessor, pointers, arrays and strings; structures, unions and enums; and I/O and files each have their own lesson in this chapter.

High-yield list

① 32 keywords (C89)
② int 4 B · char 1 B
③ -7/2 = -3 · -7%2 = -1
④ float f = 7/2 → 3.0
⑤ -1 < 1u is false (0)
⑥ b = a++ uses old a
⑦ && and || short-circuit
⑧ sizeof does not evaluate
⑨ switch: no break ⇒ fall through
⑩ do-while runs at least once

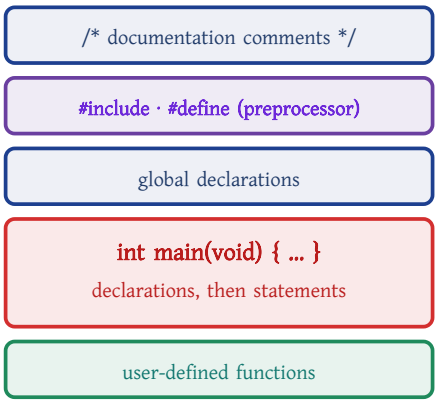
Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Program structure & tokens … p. 2
✓ Data types, sizes & ranges … p. 3
✓ Type conversion & overflow … p. 4
✓ Operators I: arithmetic to ++ … p. 5
✓ Operators II: bitwise, ?:, sizeof … p. 6
✓ Precedence & expressions … p. 7
✓ Decisions: if & switch … p. 8
✓ Loops & jump statements … p. 9
✓ Night-before p. 10 · Practice p. 11

1 Structure of a C Program & Its Tokens

Definition: C is a **structured (procedural) language**: a program is a set of functions, execution always starts at **main()**, and control flows only through **sequence, selection and iteration** — the three structures of structured programming (Böhm–Jacopini). The compiler first splits the source into **tokens**, the smallest meaningful units.



top → bottom = the usual layout

the five sections; only **main()** is compulsory

```c
#include <stdio.h>   /* header file    */
int global = 100;    /* global variable */
int main(void) {     /* starts here     */
    int local = 10;  /* local variable  */
    printf("Sum: %d\n", global + local);
    return 0;        /* 0 = success     */
}
```

Output: **Sum: 110**. `\n` is an **escape sequence** inside the string — one character, newline. `return 0;` reports success to the operating system.

Token type	What it is	Examples
Keyword	reserved word with fixed meaning, all **lowercase**	int, while, sizeof, return
Identifier	a name you choose for a variable, function, label	sum, _total, area2
Constant	a fixed value written in the source	25, 031, 0x19, 3.14f, 'A'
String literal	characters in double quotes, '\0' added	"Dhaka" (6 bytes)
Operator / punctuator	acts on operands / separates	+ ++ && ?: · ; , { } ( )

Keywords: **C89/C90: 32** · C99 adds 5 (inline, restrict, _Bool, _Complex, _Imaginary) = 37 · C11 adds 7 = 44. **Not** keywords: `main`, `printf`, `include`, `define`, `NULL`, `string`, `boolean`; `bool/true` come from <stdbool.h> (keywords only from C23).

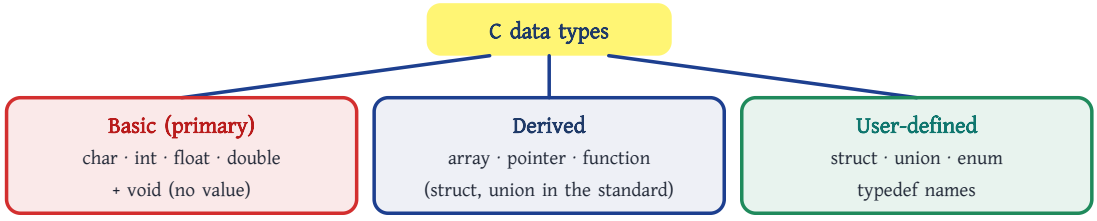
Identifier rules: Letters, digits and _ only; must **not start with a digit**; no spaces or symbols (`my-var`, `$x` `X`); not a keyword; **case-sensitive** (Sum ≠ sum). At least 31 significant characters (C89), 63 in C99.

Memory trick: Constant prefixes: "**0 = Octal, 0x = heX**". 25 = **031** = **0x19**. Suffixes: **U** unsigned, **L** long, **LL** long long, **f** float. A bare **3.14** is a **double**.

Exam trap: `int x = 010;` prints **8**, not 10 (octal). `'A'` is a character constant of type **int** in C, so **sizeof('A') = 4** (1 in C++); `"A"` is a string: **sizeof("A") = 2**. `089` is an error — 8, 9 are not octal digits.

2 Data Types, Sizes & Ranges

Definition: A **data type** fixes three things about a variable: how many bytes it occupies, the **range** of values it can hold, and which **operations** are legal on it. The standard fixes only **minimum** sizes; the actual sizes depend on the compiler and platform — always check with `sizeof`.



modifiers **short, long, signed, unsigned** adjust the basic types; struct and union are classed as derived *or* user-defined — both answers appear in exams

Type	Typical size	Range (two's complement)	printf
char / signed char	1 B	−128 … 127	%c, %d
unsigned char	1 B	0 … 255	%c, %u
short	2 B	−32,768 … 32,767	%hd
int	**4 B**	−2,147,483,648 … 2,147,483,647	%d, %i
unsigned int	4 B	0 … 4,294,967,295	%u, %x, %o
long	4 or 8 B	at least ±2,147,483,647	%ld
long long	8 B	≈ $\pm 9.22 \times 10^{18}$	%lld
float	4 B	≈ $\pm 3.4 \times 10^{38}$, **6–7** digits	%f, %e, %g
double	8 B	≈ $\pm 1.7 \times 10^{308}$, **15–16** digits	%f (scanf %lf)
long double	8–16 B	≥ double	%Lf

Data model	int	long	pointer
16-bit (Turbo C)	2	4	2
ILP32 (32-bit)	4	4	4
LP64 Linux/macOS	4	**8**	8
LLP64 Windows	4	**4**	8

Rules the standard guarantees: **sizeof(char) = 1** always · 1 ≤ short ≤ int ≤ long ≤ long long · short, int ≥ 16 bits, long ≥ 32, long long ≥ 64 · range of n bits: signed $-2^{n-1} \ldots 2^{n-1}-1$, unsigned $0 \ldots 2^{n}-1$ · limits in <limits.h> (INT_MAX) and <float.h> (FLT_DIG = 6).

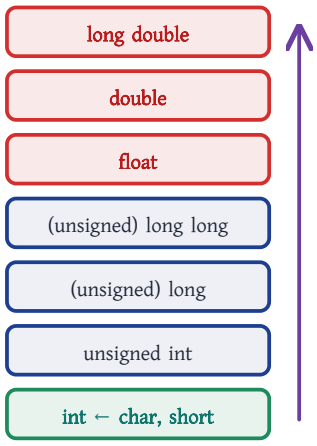
Worked example ✍ char is an integer: `char c = 'A'; printf("%d %c", c, c + 2);` → **65 C**: c + 2 is the int 67, printed as a character. '0' = 48 and 'a' − 'A' = 32.

Exam trap: `float f = 0.1;` then `f == 0.1` is **false**: 0.1 is a double, the float copy is rounded to fewer bits (0.5, exact in binary, compares true). Use `fabs(a - b) < 1e-6`.

Memory trick: "**1-2-4-8, 4-8**": char, short, int, long long; float, double. On 16-bit Turbo C int shrinks to 2 B, so "range of int on a 16-bit compiler" = **−32,768 … 32,767**.

3 Type Conversion, Casting & Overflow

Definition: **Implicit conversion** (coercion) is done by the compiler: in mixed expressions, in assignment and when passing arguments. **Explicit conversion** is a **cast**, written `(type) expression`. The type of an operation is decided by its **operands**, never by the variable that receives the result.



usual arithmetic conversions

1. **Integer promotion** first: char, short (signed or unsigned) and _Bool become **int** before any arithmetic. So `'A' + 1` is an int.
2. If either operand is **long double / double / float**, the other is converted to that type.
3. Otherwise the operand of lower **rank** rises to the higher one; at equal rank, **unsigned wins**.
4. **Assignment** converts the value to the left side's type: float → int **truncates toward zero** (3.99 → 3, −3.99 → −3).

Expression	Value	Why
`float f = 7 / 2;`	**3.0**	int ÷ int = 3, then widened
`(float)7 / 2`	**3.5**	cast first, 2 → float
`(float)(7 / 2)`	**3.0**	cast after the int division
`5 / 2 * 2.0`	**4.0**	5/2 = 2 first (left to right)
`int i = 3.99;`	**3**	fraction discarded

Overflow: **Unsigned** arithmetic never overflows — it **wraps modulo 2^n**: `unsigned char u = 255; u++;` → 0; `unsigned x = 0; x--;` → 4,294,967,295. **Signed** overflow (INT_MAX + 1) is **undefined behaviour**; most machines happen to wrap to INT_MIN. Storing an out-of-range value in a signed char is implementation-defined: `char c = 200;` usually gives **−56** (200 − 256).

Exam trap: signed vs unsigned:

```
int i = -1;  unsigned u = 1;
if (i < u) printf("less");
else       printf("greater");
```

prints **greater**: −1 is converted to unsigned = 4,294,967,295. Same reason: `sizeof(int) > -1` is **false** (sizeof gives unsigned size_t). But `unsigned char` and `unsigned short` promote to int first, so −1 < (unsigned char)1 is true.

Memory trick: "**The operation happens in the operands' type; the result is converted only on landing.**" Look at the right-hand side first: two ints ⇒ integer division, whatever type is waiting on the left.

Exam trap: `printf("%f", 5 / 2);` passes an **int** to %f — undefined behaviour (often prints 0.000000), not 2.5. `printf("%d", 2.5);` is equally wrong. Match the specifier to the type.

4 Operators I: Arithmetic to ++/--

Group	Operators	Result / note
Arithmetic	+ - * / %	% needs **integer** operands (use fmod() for floats)
Relational	< <= > >= == !=	int **1** (true) or **0** (false)
Logical	&& || !	1 or 0; any non-zero value counts as true
Assignment	= += -= *= /= %= &= |= ^= <<= >>=	`a *= b + 1` means a = a * (b + 1)
Increment/decrement	++ -- (prefix and postfix)	only on a modifiable variable (lvalue)

Integer / and % (C99 rule): Division **truncates toward zero**; the remainder takes the **sign of the dividend**, so that (a / b) * b + a % b == a always holds.

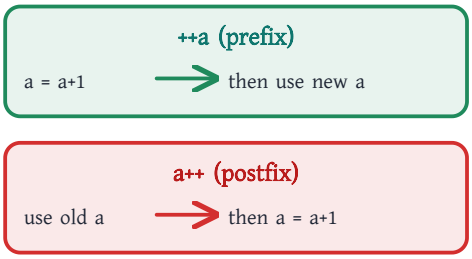
a, b	a / b	a % b
17, 5	3	2
-7, 2	**-3**	**-1**
7, -2	-3	1
-7, -2	3	-1

Integer division by zero is undefined (usually a crash).

Worked example ✍ short-circuit:

```
int a = 0, b = 5;
if (a && ++b) ;   /* ++b skipped  */
if (a || ++b) ;   /* ++b runs     */
printf("%d", b);  /* 6            */
```

&& stops at the first false operand, **||** at the first true one. Classic: `i = 1, j = 0; k = i++ || j++;` → i = 2, **j = 0**, k = 1, because i++ yielded 1 (true) and j++ never ran.



either way a ends up one larger; only the value handed to the expression differs

Worked example ✍ pre vs post:

```
int a = 5, b, c;
b = a++;    /* b = 5, then a = 6 */
c = ++a;    /* a = 7, then c = 7 */
printf("%d %d %d", a, b, c);
```

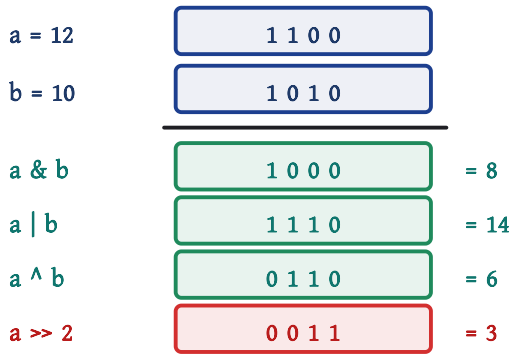
Output: **7 5 7**. As a statement on its own, `a++;` and `++a;` do the same thing.

Exam trap: (i) `10 > 5 > 2` is **0**: (10 > 5) = 1, then 1 > 2 = 0. (ii) `!5` = 0, `!!7` = 1. (iii) `5++`, `(a + b)++` are compile errors — not lvalues. (iv) `x = x++;` is undefined, see sheet 7.

Memory trick: Read the position: "**++ before = change before; ++ after = change after.**" For logic: "**AND needs one zero to quit, OR needs one non-zero to quit**" — whatever comes after that point is never evaluated.

5 Operators II: Bitwise, Shift, ?:, Comma, sizeof

Definition: **Bitwise operators** work on each bit of **integer** operands: & (AND), | (OR), ^ (XOR), ~ (one's complement, unary), << (left shift), >> (right shift). They are not allowed on float or double.



12 and 10 bit by bit; a << 2 = 48, ~a = −13

Rule	Fact
x << n	$x \times 2^n$ (if no overflow)
x >> n	$x \div 2^n$, floor, for $x \geq 0$
~x (int)	**−x − 1**: ~5 = −6, ~0 = −1
negative >>	implementation-defined
shift ≥ width	undefined behaviour

Bit tricks: **x & 1** → odd test · **x & (x − 1) == 0** → power of 2 (x > 0) · set bit k: **x | (1 << k)** · clear: **x & ~(1 << k)** · toggle: **x ^ (1 << k)** · swap: a ^= b; b ^= a; a ^= b.

Conditional (ternary) ?:: `c ? e1 : e2` — the only operator with three operands; **only one** of e1, e2 is evaluated. Right-associative: `a > b ? (a > c ? a : c) : (b > c ? b : c)` gives the largest of three. `printf("%s", 7 % 2 ? "odd" : "even");` → odd.

Comma operator: Evaluates left to right; the value is the **rightmost operand**; it has the **lowest** precedence. `x = (1, 2, 3);` → x = 3, but `x = 1, 2, 3;` → x = 1 (= binds first). Commas between function arguments or in declarations are separators, not this operator.

Worked example ✍ sizeof does not evaluate:

```
int i = 5;
int s = sizeof(i++);
printf("%d %d", i, s);   /* 5 4 */
```

sizeof is a **compile-time unary operator** (and a keyword): it needs only the type, so i++ never runs. Result type size_t, printed with **%zu**. Parentheses are needed only for a type name: `sizeof x`, `sizeof(int)`.

Exam trap: (i) Logical vs bitwise: `2 && 1` = **1** but `2 & 1` = **0**. (ii) ^ is XOR, **not** power — 2 ^ 3 = 1; use pow(). (iii) sizeof(3.14) = 8 (double), sizeof(3.14f) = 4. (iv) The pointer operators & and *, and the member operators . and ->, are covered in the pointers and structures lessons.

Worked example ✍️ count the 1-bits:

```
int n = 13, c = 0;    /* 1101 */
while (n) {
    n &= n - 1;       /* drop lowest 1 */
    c++;
}
printf("%d", c);      /* 3 */
```

13 = 1101 → 1100 → 1000 → 0000: the loop runs once per set bit, so it prints **3**. The same **n & (n - 1)** idea gives the power-of-2 test. Negative numbers are stored in **two's complement**: **-x = ~x + 1**, so in 8 bits -13 = 1111 0011.

6 Precedence, Associativity & Expressions

#	Operators (high → low)	Assoc.
1	() [] -> . postfix ++ --	L → R
2	unary + - ! ~ prefix ++ -- * & sizeof (type)	**R → L**
3	* / %	L → R
4	+ -	L → R
5	<< >>	L → R
6	< <= > >=	L → R
7	== !=	L → R
8–10	& then ^ then |	L → R
11–12	&& then ||	L → R
13	?:	**R → L**
14	= += -= *= /= %= &= ^= |= <<= >>=	**R → L**
15	, (comma)	L → R

Memory trick: "PUMA'S REBL TAC": **P**ostfix, **U**nary, **M**ultiplicative, **A**dditive, **S**hift, **R**elational, **E**quality, **B**itwise (& ^ |), **L**ogical (&& ||), **T**ernary, **A**ssignment, **C**omma. Only **unary, ternary, assignment** group right to left.

Worked example ✍️ evaluate: **(a)** `2 + 3 * 4 % 5 - 6 / 4` → 12 % 5 = 2, 6 / 4 = 1 → **3**. **(b)** a = 10, b = 20, c = 30: `a + b * c / 10 % 7` → 600 / 10 = 60, 60 % 7 = 4 → **14**. **(c)** `1 << 2 + 1` → + first → 1 << 3 = **8**. **(d)** `5 & 3 | 4` → (1) | 4 = **5**. **(e)** `a = b = c = 5` → right to left, all **5**.

Exam trap: `if (x & 1 == 0)` parses as **x & (1 == 0)** = x & 0 = **always 0** — bitwise operators sit **below** ==, so write `(x & 1) == 0`. Likewise `*p++` is ***(p++)**, and `!a + b` is (!a) + b.

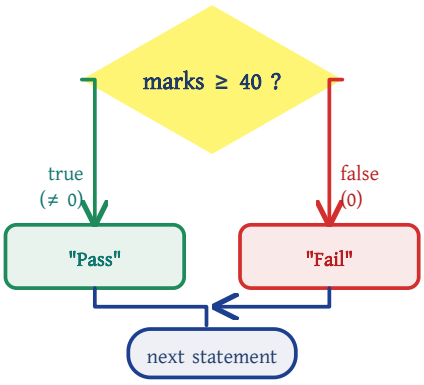
Expression · lvalue · rvalue: An **expression** is operands joined by operators that yields a value (arithmetic, relational, logical, conditional, assignment). An **lvalue** designates a storage location and may stand on the left of =; an **rvalue** is just a value. `5 = x`, `a + b = c`, `x++ = 3` are errors. An assignment is itself an expression: `while ((ch = getchar()) != EOF)`.

Sequence points & undefined behaviour: Modifying a variable **twice**, or modifying and also reading it, with no **sequence point** in between is **undefined**: `i = i++;` · `i = ++i + i++;` · `a[i] = i++;` · `printf("%d %d", i, i++);`. Sequence points: the end of a full expression (;), after the left operand of **&&, ||, ?:** and the comma operator, and before a function is called.

Exam trap: **Precedence is not order of evaluation.** In `f() + g() * h()` the multiplication is grouped first, but the compiler may **call** f, g, h in any order; function arguments too are evaluated in an unspecified order. The MCQ answer for `i = i++ + ++i` is "undefined / compiler-dependent". **gcc -Wall** warns "operation on 'i' may be undefined".

7 Decision Making: if, else-if Ladder, switch

Definition: **Selection statements** choose one path from several. A condition is any scalar expression: **zero = false, every non-zero value (even -1) = true.** Forms: **if**, **if-else**, the **else-if ladder**, **nested if**, and **switch** for many equality tests on one integer value.



if-else: exactly one branch runs

```
if (marks >= 80)      printf("A");
else if (marks >= 60) printf("B");
else if (marks >= 40) printf("C");
else                  printf("F");
```

An **else-if ladder** is tested top to bottom and stops at the **first true** condition, so order the conditions from the strictest down: marks = 85 prints only **A**.

Worked example ✍ dangling else:

```
int a = 5, b = 10;
if (a > 10)
    if (b > 5) printf("X");
else printf("Y");
```

Prints **nothing**: an else belongs to the **nearest unmatched if**, whatever the indentation says. Use braces.

Exam trap: (i) `if (x = 0)` **assigns** `0` → always false; `if (x = 10)` → always true and x becomes 10. (ii) `if (x == 5);` — the stray semicolon is an empty body, so the next line always runs. (iii) Writing `5 == x` makes the typo `5 = x` a compile error.

switch rules: The expression must be an **integer type** (int, char, enum) — no float, no strings. Case labels are **integer constant expressions**, unique, no ranges. **default** is optional and may appear anywhere. Without **break**, control **falls through** to the next case.

Worked example ✍️ fall-through:

```
int n = 2;
switch (n) {
  case 1: printf("A");
  case 2: printf("B");
  case 3: printf("C"); break;
  default: printf("D");
}
```

Jumps to case 2, prints B, falls into case 3, prints C, then break → **BC**.

Worked example ✍️ default first:

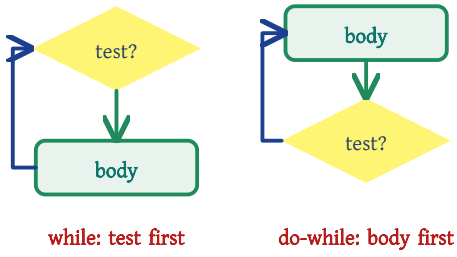
```
int n = 5;
switch (n) {
  default: printf("D");
  case 1:  printf("A"); break;
  case 2:  printf("B");
}
```

No case matches, so control goes to default **wherever it is**: D, then falls into case 1: A, break → **DA**.

Memory trick: "**switch is a lift, not a staircase**": it jumps straight to the matching floor and then keeps going down until a **break** stops it. Use **switch** for one integer compared with constants (often compiled to a fast jump table); use an **if ladder** for ranges, floats, strings or mixed conditions.

8 Loops (Iteration) & Jump Statements

Loop	Test	Body runs
`while (c) S`	at entry	0 or more times
`for (i; c; u) S`	at entry	0 or more; parts optional, empty c = true
`do S while (c);`	at exit	**at least once**



entry vs exit control

Worked examples ✍️ trace these outputs:

```
/* (a) */ for (i = 0; i < 5; i++);
          printf("%d", i);        /* 5    */
/* (b) */ for (i = 1; i <= 100; i *= 2)
              count++;            /* 7    */
/* (c) */ i = 10;
          do printf("%d", i);
          while (i < 5);          /* 10   */
/* (d) */ n = 3;
          while (n--) printf("%d", n);
                                  /* 210  */
```

```
/* (e) */ i = 0;
while (i < 5) {
    if (i == 2) { i++; continue; }
    printf("%d", i);  i++;
}                          /* 0134 */
/* (f) */
for (i = 1; i <= 3; i++)
  for (j = 1; j <= 3; j++) {
    if (j == 2) break;
    printf("%d%d ", i, j);
  }                        /* 11 21 31 */
```

(a) the **;** after for is an empty body, so only the final i = 5 prints. (b) i = 1, 2, 4, 8, 16, 32, 64 → **7 passes**. (c) do-while prints once although 10 < 5 is false. (d) the test uses the old n, the body sees the new one; n ends at -1. (f) **break** leaves only the **innermost** loop.

Jump	Effect	Watch out
break	exits the innermost loop or switch	not allowed in a bare if
continue	skips to the next iteration (loops only)	in for, jumps to the **update**; in while, to the **test**
goto label	unconditional jump within the same function	legitimate use: escaping nested loops, cleanup
return	leaves the function (functions lesson)	`exit()` is a library function, not a jump statement

Exam trap: infinite loops: (i) `while (i < 5) { if (i == 2) continue; i++; }` — continue skips the i++, so i stays 2 for ever. (ii) `for (unsigned u = 10; u >= 0; u--)` never ends: an unsigned value is **always ≥ 0**. (iii) `for (;;)` and `while (1)` are deliberate infinite loops.

Memory trick: "**do-while is the waiter who serves first and asks later.**" Count iterations as **(last - first) / step + 1** for additive loops: for (i = 0; i < 10; i += 3) → 0, 3, 6, 9 → 4. `for (int i = 0; …)` needs **C99**; old C89 compilers reject it.

Night-Before-the-Exam Sheet

Fact sheet:

32 keywords (C89), 37 (C99)

010 = 8 · 0x1F = 31

char 1 · short 2 · int 4 · long long 8

long: 8 on LP64, 4 on Windows

char, short → int first

float → int truncates to 0

-7 / 2 = -3 · -7 % 2 = -1

~x = -x - 1

comma: value of the right

else → nearest if

identifier: not a digit first

sizeof('A') = 4 in C

float 4 (6–7 dig.) · double 8 (15–16)

16-bit int: -32,768 … 32,767

equal rank: unsigned wins

unsigned wraps mod 2^n

% sign = sign of dividend

x << n = x × 2^n

sizeof: not evaluated

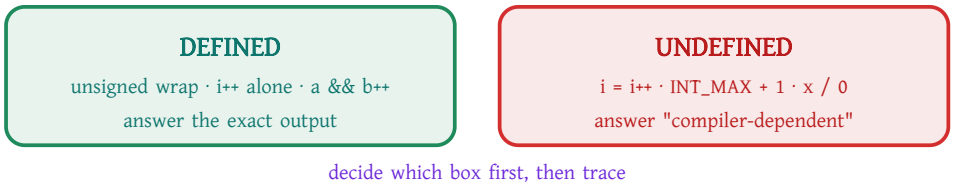
switch: integer only

Top 12 one-liners for MCQs

1. `float f = 7/2;` → **3.0**
2. `-1 < 1u` → **0** (false)
3. `b = a++` takes the **old** a
4. `k = i++ || j++` → j++ skipped
5. `x & 1 == 0` → x & 0
6. `i = i++` → **undefined**
7. Only unary, ?:, = are R → L
8. Lowest precedence: **comma**
9. No break ⇒ **fall through**
10. default may be anywhere
11. do-while: **at least once**
12. break leaves the **innermost** loop

Top traps: the left-hand type does not change the operation (7/2 is still 3) · ^ is XOR, not power · 2 && 1 = 1 but 2 & 1 = 0 · float == float is unreliable · unsigned u >= 0 is always true · `if (x = 0)` assigns · a stray `;` after for/if/while empties the body · precedence ≠ order of evaluation · `printf("%f", 5/2)` is undefined.

All tricks together: "0 = Octal, 0x = heX" · "1-2-4-8, 4-8" · "the operation happens in the operands' type; the result converts on landing" · "++ before = change before" · "AND quits on a zero, OR quits on a non-zero" · PUMA'S REBL TAC · "switch is a lift, not a staircase" · "do-while serves first and asks later".



the first question to ask of every "what is the output" snippet

✓ Practice MCQs (exam pattern, 10 marks)

1. sizeof(int) with gcc, 64-bit Linux:
(a) 2 (b) 4
(c) 8 (d) 16

2. printf("%d", -7 / 2) prints:
(a) -4 (b) 3
(c) -3 (d) -3.5

3. NOT a keyword in C:
(a) sizeof (b) volatile
(c) goto (d) string

4. a = 5; b = a++ + 2; b is:
(a) 7 (b) 8
(c) 6 (d) undefined

5. The value of -1 < 1u is:
(a) 1 (b) 0
(c) error (d) -1

6. Body runs at least once:
(a) do-while (b) for
(c) while (d) none

7. Lowest precedence:
(a) = (b) ?:
(c) , (comma) (d) ||

8. sizeof('A') in C (int = 4 B):
(a) 4 (b) 1
(c) 2 (d) 8

9. The value of 5 & 3 | 4 is:
(a) 7 (b) 1
(c) 4 (d) 5

10. int x = 010; printf("%d", x);
(a) 10 (b) 8
(c) 2 (d) 16

Answer key → 1-b · 2-c · 3-d · 4-a · 5-b · 6-a · 7-c · 8-a · 9-d · 10-b

Written question: (a) Distinguish **while** and **do-while** with a flowchart each. (b) Trace the output of the program below. (c) Write a C program that reads an integer and prints the sum of its digits. 10

```
int i, s = 0;
for (i = 1; i <= 5; i++)
    switch (i % 3) {
        case 0:  s += 10; break;
        case 1:  s += 1;
        default: s += 2;
    }
printf("%d", s);
```

1. **(a)** while is **entry-controlled**: test, then body, so it may run 0 times. do-while is **exit-controlled**: body, then test, so it runs at least once and ends with `while (c);`. Flowcharts as on sheet 9. 3
2. **(b)** i = 1: case 1 → +1, falls into default → +2, s = 3 · i = 2: default → 5 · i = 3: case 0 → 15 · i = 4: +1 +2 → 18 · i = 5: +2 → **20**. Output **20**. 3
3. **(c)** `int n, sum = 0; scanf("%d", &n); if (n < 0) n = -n; while (n > 0) { sum += n % 10; n /= 10; } printf("%d", sum);` — n % 10 takes the last digit, n /= 10 drops it; 4731 → 1 + 3 + 7 + 4 = 15. 4

Before the exam ✓

- ✓ Can state size and range of each basic type
- ✓ Can apply the usual arithmetic conversions
- ✓ Can parse any expression with PUMA'S REBL TAC
- ✓ Can trace switch fall-through and loops